Republic of the Philippines
The National Teachers College
School of Arts, Sciences, and Technology

Nueva Ecija Offline Agricultural Decision Support System (Agri-DSS): A Standalone Decision System for Precision Rice Management in Connectivity-Independent Zones

Target SDG: 2

Final Terminal Project

Submitted by:
Genei Ryodan
DELCASTILLO, Vicente Jr. C.
MANALO, Ronn Gabrielle B.
PATDO, Verill Jovan T.
VIRAY, Lei Matthew A.
1.3BSIT

Submitted to:
Miss Justin Louise R. Neypes

Date:
May 2026

The National Teachers College

I. INTRODUCTION

A. Project Overview & Target SDG

Nueva Ecija Offline Agricultural Decision Support System (Agri-DSS): A Standalone Decision System for Precision Rice Management in Connectivity-Independent Zones is a C++ console application built to help small-scale rice farmers in Nueva Ecija make timely, informed decisions about crop management. The system collects daily temperature and rainfall inputs from the user, computes Growing Degree Days (GDD) to track the biological development of the rice crop, and generates specific recommendations across four areas: crop stage identification, fertilizer application, irrigation scheduling, and harvest readiness. Farm records are stored in a local MySQL database and are restored automatically at startup so the system maintains continuity across multiple sessions. When the database connection is unavailable, the system runs in Offline Mode using session data held in memory. While the application is designed to run on everyday computers, it requires a one-time technical setup to configure the local MySQL environment prior to field deployment. Farmers with inconsistent internet access can still receive full guidance.

The project responds directly to **Sustainable Development Goal 2: Zero Hunger**, which calls for ending hunger, achieving food security, and promoting sustainable agricultural practices, particularly for smallholder and subsistence farmers. Rice farming in Nueva Ecija carries real financial risk at every stage of the crop cycle. Applying nitrogen fertilizer just before heavy rainfall wastes inputs and pollutes nearby water systems. Waiting too long to harvest a mature crop during wet weather leads to grain shattering and rotting. Farmers dealing with these situations rarely have immediate access to agronomists or online tools. The Agri-DSS puts a

working decision-support system on a basic computer and translates observable weather conditions into concrete farm actions. Preventing those specific, avoidable losses is a direct contribution to what SDG 2 is asking for at the farm level.

B. Problem Statement

Small-scale rice farmers in Nueva Ecija frequently manage their fields under unpredictable micro-climates, relying on generational experience to time critical, cost-heavy interventions like fertilizer application and harvesting. While modern agricultural science utilizes metrics like Growing Degree Days to optimize these exact operational windows, that analytical power is typically locked behind cloud-based applications or complex agricultural extensions. Out in the rural field, internet connectivity is often inconsistent or entirely unavailable. Farmers have the raw observations, they know the day's temperature and the rainfall level, but they lack a fast, reliable way to translate those inputs into stage-specific agronomic decisions.

When farming decisions are made without this translation, the financial and physical risks are immediate. Applying nitrogen right before a heavy downpour leads to chemical runoff, wasting expensive fertilizer inputs and threatening local water systems. Similarly, delaying a harvest when the crop is physiologically mature and rain is imminent introduces a severe risk of grain shattering and rotting. The Nueva Ecija Offline Agri-DSS addresses this directly. It processes simple user inputs through established agricultural thresholds, no internet connection required, and gives farmers immediate, stage-specific guidance from a basic computer. That is what the system exists to do.

II. REQUIREMENT ANALYSIS

A. Functional Requirements (FR)

FR ID	Requirement	Rubric Definition	System Implementation (Evidence from Code)	Source Files / Functions
FR1	**Automated Data Initialization**	Upon execution, the system must automatically detect and connect to an external database. It must parse a minimum of 20 pre-existing records (the system implements a 30-record window, which satisfies this threshold) and load them into an internal session collection.	1. On startup, main.cpp calls db.connect() to establish a connection to the external MySQL database (agri_dss). 2. If the connection succeeds, getTotalGDD() is called to reconstruct the accumulated GDD total from all stored daily_gdd values via a SUM() query. 3. loadRecentRecords() then fetches up to 30 records from the daily_records table and populates the FarmSession arrays (recordIds[], temps[], rainfall[], daysRecorded) before the menu appears. These arrays, held inside the FarmSession class, serve as the in-memory session collection for all advisory modules. 4. validateSchema() runs automatically inside connect() and confirms that all four required columns (id, temperature, rainfall_mm, daily_gdd) exist before any data is read. 5. If the database is unavailable or schema validation fails, the system enters Offline Mode automatically and continues with empty session arrays. 6. When the 30-record session limit is reached during inputFarmData(), the system shifts all existing array entries left by one position to make room for the new entry, maintaining a rolling	main.cpp (startup block); DatabaseManager.cpp: connect(), getTotalGDD(), loadRecentRecords(), validateSchema(); agriManager.cpp: loadFromTextFile()

			window of the most recent 30 days. This shift only affects the in-memory session arrays. Database records are not modified by the shift and can only be removed through the season reset function (menu option 8). 7. Before the database path runs, main.cpp calls loadFromTextFile() in agriManager.cpp, which attempts to read INPUT_DATA/database.txt and pre-seed the session arrays with any records stored there. Each line in the file is parsed as a comma-separated entry containing a record ID, temperature, rainfall, and daily GDD value. If the file is not found, the system prints a diagnostic message and continues with an empty session. This ensures that data from a prior offline session is available even before the MySQL connection is attempted.	
FR2	**Secure Record Management (CRUD)**	**CREATE:** Users must input new data entries. **READ:** The system must provide a View All summary and targeted, stage-specific data outputs. **UPDATE:** Users must modify existing record attributes. **DELETE (Targeted and Complete):** The system provides	1. CREATE: inputFarmData() in agriManager.cpp collects validated temperature and rainfall input, appends the new entry to the FarmSession rolling arrays, and accumulates GDD. Upon successful input, main.cpp immediately calls insertDailyRecord() to persist the new row to daily_records. 2. READ (View All): displaySummaryReport() prints a full tabular listing of all entries in the rolling window, including temperature, rainfall, total days, and accumulated GDD. 3. READ (Specific Search): Option 6 in the main menu calls getRecordById() within	agriManager.cpp: inputFarmData(), displaySummaryReport(), all Module::run() methods; DatabaseManager.cpp: insertDailyRecord(), resetDatabase(), getRecordById(), updateDailyRecord(), deleteRecord();

		two distinct deletion methods to ensure secure record management. Targeted deletion (menu option 9) allows users to remove a single historical entry by its unique database ID, executing deleteRecord() within DatabaseManager.cpp. Complete deletion, or the Reset function (menu option 8), zeroes all in-memory session arrays and calls resetDatabase(), which executes a TRUNCATE TABLE daily_records command to remove all persisted records in preparation for a new crop season.	DatabaseManager.cpp. This executes a parameter-bound SELECT query, allowing users to retrieve and view a specific historical record using its unique database ID. 4. DELETE: The Reset function (menu option 8) zeroes all in-memory session arrays and calls resetDatabase(), which executes TRUNCATE TABLE daily_records to remove all persisted records for a new season. 5. UPDATE: Option 7 in the main menu allows users to modify existing entries. It captures new temperature and rainfall inputs, recalculates the daily GDD, and executes an UPDATE query via updateDailyRecord(). Immediately after a successful database update, the system calls loadRecentRecords() to refresh the local session arrays, guaranteeing memory synchronization. 6. DELETE (Targeted): Option 9 in the main menu prompts the user for a Record ID and calls deleteRecord() in DatabaseManager.cpp, which executes a parameterized DELETE query against daily_records. Following the deletion, the system reloads the session arrays via loadRecentRecords() and calls saveToTextFile() to sync the backup file, guaranteeing memory and file consistency.	main.cpp: case 1, case 6, case 7, case 8, case 9
FR3	**Core Domain Logic (Computation Engine)**	The system must implement a Value-Added calculation	1. GDD Accumulation Engine: inputFarmData() computes the daily Growing Degree Day (GDD) increment using the formula	agriManager.cpp : inputFarmData() (GDD engine),

		module. It must not just store data but process it, such as calculating a derived metric or predicting outcomes based on stored values.	(temperature - 10.0°C) and adds it to session.totalGDD. This transforms raw temperature readings into a biologically meaningful crop development metric. 2. Crop Stage Classifier (CropStageModule): Uses accumulated GDD to classify the crop into Vegetative (0 to less than 400 GDD), Reproductive (400 to less than 1000 GDD), or Ripening (1000 GDD and above) stages. 3. Fertilizer Decision Engine (FertilizerModule): Applies compound logic combining GDD stage and the most recent rainfall value to determine whether applying fertilizer will result in runoff waste, is appropriate, or must be withheld to protect grain quality. 4. Irrigation Dry-Spell Detector (IrrigationModule): Counts consecutive dry days (rainfall < 2mm) backwards from the most recent entry and triggers a 30mm irrigation alert if a 3-day dry streak or a temperature above 35°C is detected. 5. Harvest Readiness Predictor (HarvestModule): Cross-references GDD against maturity thresholds (1000 GDD, 1200 GDD) and current rainfall to predict harvest timing and issue critical grain-loss alerts when conditions are unfavorable.	CropStageModule::run(), FertilizerModule::run(), IrrigationModule::run(), HarvestModule::run()
FR4	**Persistent State Storage**	The system must synchronize internal memory with the external file. All additions and deletions	1. Every successful call to inputFarmData() in Online Mode immediately triggers insertDailyRecord(), which writes the day's temperature, rainfall_mm, and daily_gdd increment as a single row to	DatabaseManager.cpp: insertDailyRecord(), resetDatabase(), getTotalGDD(),

		must be persisted to the external database. The system writes each new record immediately after entry rather than batching saves on exit, which satisfies the persistence requirement and reduces the risk of data loss mid-session.	the MySQL daily_records table. Following any database modification that adds, updates, or removes individual records (insert, update, or targeted delete), the system actively refreshes the session arrays and calls saveToTextFile() to create a redundant offline backup in database.txt. The full season reset (menu option 8) wipes the MySQL table via TRUNCATE and zeroes the in-memory session, but does not rewrite database.txt, as the file is refreshed on the next session's startup load. In Offline Mode, data is held in memory only and insertDailyRecord() is not called. 2. On startup, the system restores all prior session data from the database via getTotalGDD() and loadRecentRecords(), ensuring continuity across program runs without manual re-entry. 3. Season reset (menu option 8) calls resetDatabase(), which executes TRUNCATE TABLE daily_records to wipe all persisted records in sync with the in-memory session wipe. 4. If a database error occurs mid-session, the system catches the exception, invalidates the connection, and continues in Offline Mode. In-memory data remains usable for the rest of the session. 5. On exit (menu option 10), the summary report is displayed. If the session ran in Online Mode, the program confirms that data is preserved in the database. If the session ran in Offline Mode, in-memory data was not persisted and	loadRecentRecords(); main.cpp: case 1 (insert after entry), case 8 (reset), case 9 (delete), case 10 (exit confirmation)

			the exit confirmation message did not apply.	
FR5	**Navigational Menu System**	A robust, looped menu interface must allow users to transition between modules (in this system: Input, Analyze, Recommend, Schedule, Predict, Search, Edit, Reset, Delete, and Exit) without program termination.	1. displayMenu() in agriManager.cpp prints a 10-option console menu on every iteration, including the current system time, advisory modules, search, edit, reset, and exit options. 2. The main loop in main.cpp is a do-while structure that continues until the user selects option 10 (Exit), keeping the program alive across unlimited module transitions. 3. Non-numeric input is caught via cin.fail() inside the loop. The input buffer is cleared with clearInputBuffer(), an error message is shown, and the menu re-displays without crashing. 4. End-of-file input (cin.eof()) triggers a controlled emergency shutdown with a diagnostic message rather than an unhandled exception. 5. Menu options 2 through 5 route to the advisory modules via polymorphic dispatch (modules[choice - 2]->run(session)), keeping the navigation logic minimal and extensible.	agriManager.cpp : displayMenu(), clearInputBuffer(); main.cpp: do-while loop, switch statement, cin.fail() and cin.eof() guards

B. Non-Functional Requirements (NFR)

NFR ID	Requirement	Rubric Definition	System Implementation (Evidence from Code)	Source Files / Functions
NFR1	**Robustness and Defensive Programming**	Input Validation: The system must handle Type Mismatch errors without crashing or entering infinite loops. Graceful Failure: Informative error messages must guide the user when invalid operations are attempted.	1. cin.fail() is checked after every numeric read in inputFarmData(). On failure, clearInputBuffer() flushes the stream, an error message identifying the problem is printed, and the input variable is reset to force the validation loop to retry. 2. Temperature input is restricted to the range 15.0°C to 55.0°C via a do-while loop. Any out-of-range value triggers a domain-specific error message before re-prompting. 3. Rainfall category selection is restricted to 0-3 via a do-while loop with the same fail-safe pattern. 4. cin.eof() is detected separately from cin.fail() in all input loops and in the main menu loop. It triggers a controlled shutdown with a diagnostic message rather than an infinite retry loop. 5. All database operations are wrapped in try-catch(sql::SQLException) blocks. Errors print a diagnostic message, invalidate the connection pointer, and switch the session to Offline Mode without crashing the program. 6. The confirmation prompt for season reset accepts only 1 or 0. An invalid input cancels the operation with a clear message, preventing accidental data loss.	agriManager.cpp: inputFarmData() (all input loops); main.cpp: main menu cin.fail()/cin.eof() guards, case 8 confirmation; DatabaseManager.cpp: all try-catch blocks

NFR2	**Structural Modularity**	Multi-File Separation: The project must be partitioned into .h files (declarations) and .cpp files (implementations). Encapsulation: All class attributes must be private, accessed only via public getter and setter methods.	1.The project consists of six separate files organized across four logical components: dailyRecord.h (data class), agriManager.h and agriManager.cpp (advisory module hierarchy and free functions), DatabaseManager.h and DatabaseManager.cpp (database bridge), and main.cpp (entry point and program loop). main.cpp has no paired header file, as it serves solely as the program entry point. 2. DatabaseManager.h declares the class interface with all attributes private. The sql::Connection* con pointer is hidden from all other translation units and is accessible only through the class's own public methods. 3. The private validateSchema() method is declared in DatabaseManager.h and defined in DatabaseManager.cpp. It is called internally by connect() and is not exposed to main.cpp or any other module. 4. isConnected() serves as the public read-only accessor for the connection state, returning a bool without exposing the raw pointer. 5. The AdvisoryModule base class in agriManager.h isolates the checkDataAvailable() guard in a protected scope, making it available only to derived classes and not to external callers. 6. FarmSession in dailyRecord.h was converted from a plain struct into a fully encapsulated class to satisfy the private-attribute requirement directly.	dailyRecord.h, agriManager.h, agriManager.cpp, DatabaseManager.h, DatabaseManager.cpp, main.cpp

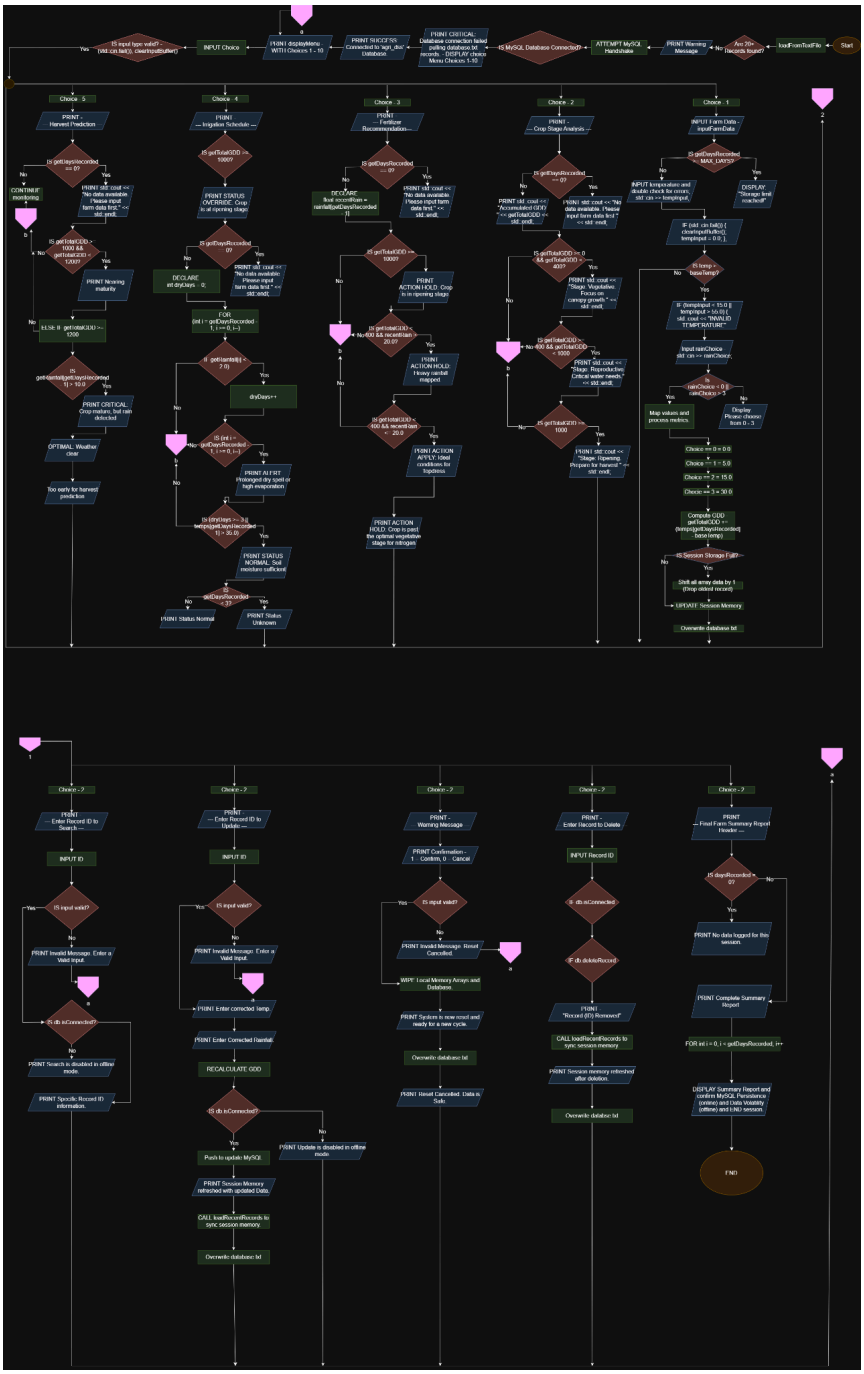
			All data members (recordIds[], rainfall[], temps[], daysRecorded, totalGDD) are private. A constructor initializes all arrays and counters to zero on object creation. All external access goes through public getters and setters, with no direct member access permitted from outside the class.	
NFR3	**Readability and Professional Documentation**	Naming Conventions: Adhere to PascalCase for class names and camelCase for variables and function names. In-Code Documentation: Every class and function must be preceded by a comment block detailing its purpose, inputs, and outputs.	1. All class names follow PascalCase: AdvisoryModule, CropStageModule, FertilizerModule, IrrigationModule, HarvestModule, DatabaseManager, FarmSession. 2. All function and variable names follow camelCase: inputFarmData(), displayMenu(), clearInputBuffer(), insertDailyRecord(), getTotalGDD(), loadRecentRecords(), daysRecorded, totalGDD, rainMapped. 3. Every function in DatabaseManager.cpp is preceded by a multi-line comment block describing its purpose, the contract it upholds, and any resource safety guarantees (e.g., explaining why pstmt is deleted in both the normal path and the exception path). 4. Every derived module class in agriManager.h includes a comment block above its declaration stating what it analyzes and what logic it applies. 5. The polymorphic module registry in main.cpp includes an indexed comment block explaining the mapping between menu choices and array indices, and why the virtual destructor is required.	DatabaseManager.h, DatabaseManager.cpp (all function headers), agriManager.h (all class headers), agriManager.cpp (inline comments), main.cpp (module registry and loop comments)

			6. FarmSession in dailyRecord.h strictly adheres to OOP encapsulation rules. It is implemented as a secure class where the rainfall and temps arrays, as well as the counters, are marked private. A constructor automatically initializes memory safely upon creation, and all data access by the advisory modules is routed through secure, public getter and setter methods. 7. main() in main.cpp and the short utility functions displayMenu() and clearInputBuffer() in agriManager.cpp carry inline comments explaining their purpose. Full formal comment blocks with documented inputs and outputs are applied to all non-trivial functions across the codebase.	
NFR4	**Memory and Resource Efficiency**	Type Selection: Use appropriate data types to optimize memory usage. Scope Management: Variables must be declared in the narrowest possible scope to ensure efficient memory lifecycle management.	1. float is used for all temperature, rainfall, and GDD values (session.temps[], session.rainfall[], session.totalGDD, dailyGDD), which are continuous measurements that do not require double precision. 2. int is used for all counters and categorical values (session.daysRecorded, rainChoice, daysRecorded loop index, confirm), matching the rubric's guidance on appropriate type selection. 3. All database resource pointers (sql::PreparedStatement* pstmt, sql::Statement* stmt, sql::ResultSet* res) are declared inside the try block of each function, giving them the narrowest possible scope. They are deleted immediately after use in the	dailyRecord.h (float/int type choices), DatabaseManager.cpp (scoped pstmt/stmt/res, destructor), agriManager.cpp (scoped locals), main.cpp (cleanup loop, scoped case variables)

			normal path and in the catch block on the error path. 4. The choice variable in main.cpp is sczoped to the do-while loop context. Local variables like gddBefore in case 1, and the temporary reloading arrays (tempIds, tempTemps, tempRain) in cases 1, 7, and 9, are declared tightly inside their respective switch case blocks using braces, limiting their lifetime to that single operation. 5. The sql::Connection* con pointer in DatabaseManager is heap-allocated via the MySQL driver and is explicitly deleted in both the destructor and in any error path that invalidates the connection, preventing memory leaks on all exit paths. 6. The four AdvisoryModule-derived objects are allocated on the heap at startup, stored in a fixed array of four pointers, and explicitly deleted in a cleanup loop before main() returns, verified through the virtual destructor chain.	

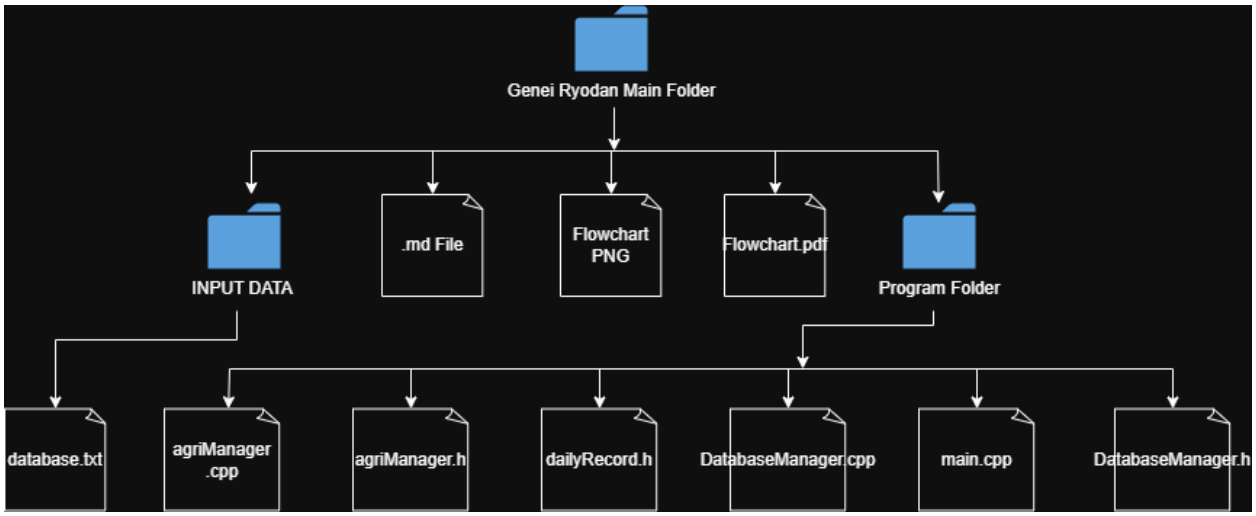
III. DESIGN SPECIFICATION

A. Algorithmic Flowchart



https://drive.google.com/file/d/16izaHYuLZk5GdskeN7KQecmVEXEc7bgF/view?usp=sharing

B. File Structure Diagram



Component Breakdown

- **main.cpp:** The entry point of the program. It starts the database connection, initializes the session, registers all four advisory modules behind base-class pointers, and runs the main menu loop until the user exits.

- **dailyRecord.h:** Defines the FarmSession class and the MAX_DAYS constant (30). Every other file in the project depends on this class to share temperature, rainfall, and GDD data across modules through its public getter and setter interface.

- **agriManager.h:** Declares the AdvisoryModule abstract base class and its four derived classes: CropStageModule, FertilizerModule, IrrigationModule, and HarvestModule. It also declares the free functions used for menu display, input handling, and session reporting.

- **agriManager.cpp:** Contains the full implementations of all four advisory modules and the free functions declared in agriManager.h. This is where the crop stage classification, fertilizer decision logic, irrigation dry-spell detection, and harvest readiness prediction are all computed.

- **DatabaseManager.h:** Declares the DatabaseManager class interface. The MySQL connection pointer is kept private here, meaning no other part of the program can access or misuse it directly.

- **DatabaseManager.cpp:** Implements all database operations: connecting to the agri_dss MySQL database, validating the table schema on startup, inserting daily records, loading recent records into the session arrays, summing accumulated GDD, searching a specific record by ID, updating an existing record by ID, securely deleting a specific record by its ID, and resetting the table at the start of a new crop cycle.